Fix Sender Algorithm Customization

Document #:	P3826R3 [Latest] [Status]
Date:	2026-01-05
Project:	Programming Language C++
Audience:	SG1 Concurrency and Parallelism Working Group LEWG Library Evolution Working Group LWG Library Working Group
Reply-to:	Eric Niebler <eric.niebler@gmail.com>

Contents

§ 1 Background

In the current Working Draft, 33 [exec] has sender algorithms that are customizable. While the sender/receiver concepts and the algorithms themselves have been stable for several years now, the customization mechanism has seen a fair bit of recent churn. [P3718R0] is the latest effort to shore up the mechanism. Unfortunately, there are gaps in its proposed resolution. This paper details those gaps.

The problem and its solution are easy to describe, but the changes are not trivial. The fix has been implemented twice, largely independently and causing no bug reports. This paper proposes to fix the issue for C++26.

§ 2 Revision history

§ 2.1 R3

- Addresses feedback from the LEWG review of R2 (see Addressing feedback from LEWG design review).
  - Adds a `get_completion_domain<>(attrs, env...)` query that defaults to `get_completion_domain<set_value_t>(attrs, env...)`.
- Remove all discussion of deferring algorithm customization to C++29.

§ 2.2 R2

— Adds `indeterminate_domain<>`.

§ 2.3 R1

— Adds an alternative resolution that suggests how to fix algorithm customization for C++26.

§ 2.4 R0

— Initial revision that proposes deferring algorithm customization to C++29.

§ 3 The problem with P3718

[P3718R0] identifies real problems with the status quo of sender algorithm customization. It proposes using information from the sender about where it will *complete* during "early" customization, which happens when a sender algorithm constructs and returns a sender; and it proposes using information from the receiver about where the operation will *start* during "late" customization, when the sender and the receiver are connected.

The problem with this separation of responsibilities is:

Many senders do not know where they will complete until they know where they will be started.

A simple example is the `just()` sender; it completes inline wherever it is started. The information about where a sender will start is not known during early customization, when the sender is being asked for this information.

And even if we knew where the sender will start, there is no generic interface for asking a sender where it will complete *given where it will start*. There currently is no such API, which is the whole problem in a nutshell.

§ 3.1 An illustrative example

This section illustrates the above problem by walking through the algorithm selection process proposed by P3718. Consider the following example:

```
namespace ex = std::execution;
auto sndr = ex::starts_on(gpu, ex::just()) | ex::then(fn);
std::this_thread::sync_wait(std::move(sndr));
```

… where `gpu` is a scheduler that runs work (unsurprisingly) on a GPU.

`fn` will execute on the GPU, so a GPU implementation of `then` should be used. By the proposed resolution of P3718, algorithm customization proceeds as follows:

— During early customization, when `starts_on(gpu, just()) | then(fn)` is executing, the `then` CPO asks the `starts_on(gpu, just())` sender where it will complete as if by:

```
auto& [_, fn, child1] = *this; // *this is a then sender
                               // child1 is a starts_on
        sender
auto dom1 = ex::get_domain(ex::get_env(child1));
```

— The `starts_on` sender will in turn ask the `just()` sender, as if by:

```
auto& [_, sch, child2] = *this;   // *this is a starts_on
        sender
                                  // child2 is a just sender
auto dom2 = ex::get_domain(ex::get_env(child2));
```

As discussed, the `just()` sender doesn't know where it will complete until it knows where it will be started, but that information is not yet available. As a result, `dom2` ends up as `default_domain`, which is then reported as the domain for the `starts_on` sender. That's incorrect. The `starts_on` sender will complete on the GPU.

— The `then` CPO uses `default_domain` to find an implementation of the `then` algorithm, which will find the default implementation. As a result, the `then` CPO returns an ordinary `then` sender.

— When that `then` sender is connected to `sync_wait`'s receiver, late customization happens. `connect` asks `sync_wait`'s receiver where the `then` sender will be started. It does that with the query `get_domain(get_env(rcvr))`. `sync_wait` starts operations on the current thread, so the `get_domain` query will return `default_domain`. As with early customization, late customization will also not find a GPU implementation.

The end result of all of this is that a default (which is effectively a CPU) implementation will be used to evaluate the `then` algorithm on the GPU. That is a bad state of affairs.

§ 4 Solutions considered

Here is a list of possible ways to address this problem for C++26, sorted by descending awfulness.

§ 4.1 Remove all of the C++26 `std::execution` additions

Although the safest option, I hope most agree that such a drastic step is not warranted by this issue. Pulling the `sender` abstraction and everything that depends on it would result in the removal of:

— The sender/receiver-related concepts and customization points, without which the ecosystem will have no shared async abstraction, and which will set back the adoption of structured concurrency three years.

— The sender algorithms, which capture common async patterns and make them reusable,

— `execution::counting_scope` and `execution::simple_counting_scope`, and related features for incremental adoption of structured concurrency,

— `execution::parallel_scheduler` and all of its related APIs, and

— `execution::task` and `execution::task_scheduler` (C++26 will *still* not have a standard coroutine task type <*heavy sigh*>).

This option should only be considered if all the other options are determined to have unacceptable risk.

§ 4.2 Remove all of the customizable sender algorithms

This option would keep all of the above library components with the exception of the customizable sender algorithms:

— `then`, `upon_error`, `upon_stopped`

— `let_value`, `let_error`, `let_stopped`

— `bulk`, `bulk_chunked`, `bulk_unchunked`

— `starts_on`, `continues_on`, `on`

— `when_all`, `when_all_with_variant`

— `stopped_as_optional`, `stopped_as_error`

— `into_variant`

— `sync_wait`

— `affine_on`

This would leave users with no easy standard way to start work on a given execution context, or transition to another execution context, or to execute work in parallel, or to wait for work to finish.

In fact, without the `bulk` algorithms, we leave no way for the `parallel_scheduler` to execute work in parallel!

While still delivering a standard async abstraction with minimal risk, the loss of the algorithms would make it *just* an abstraction. Like coroutines, adoption of senders as an async *lingua franca* will be hampered by lack of standard library support.

§ 4.3 Remove sender algorithm customization

In this option, we ship everything currently in the Working Draft but remove the ability to customize the algorithms. This gives us a free hand to design a better customization mech-

anism for C++29 – provided we have confidence that those new customization hooks can be added without break existing behavior.

This option is not as low-risk as it may seem. Firstly, it is difficult to be confident that algorithm customization can be added back without breaking code. Improved customization hooks have been implemented, and wording for the removal has been written such that that the new hooks can be standardized without breaking changes, to the best of the author's ability.

Secondly, algorithm customizability is a load-bearing feature. Taking it out is not hard but it isn't trivial either. Customizability is used by the `parallel_scheduler` to accelerate the `bulk` family of algorithms. Although the `task_scheduler` does not currently customize `bulk`, it should. Some design work is necessary before algorithm customization can be removed.

§ 4.4 Ship everything as-is and fix algorithm customization in a DR

This option is not as reckless as it sounds. We have a fix and the fix has been implemented in a working and publicly available `execution` library (CCCL). It would not be the first time the Committee shipped a standard with known defects, and the DR process exists for just this purpose.

One potential problem is that, as DRs go, this one would be large-ish. I do not know if this presents a problem procedurally. If it does, then fixing the problem now would make more sense. Any future DRs are likely to be smaller.

§ 4.5 Fix algorithm customization now

This is the option this paper proposes. The fix is easy to describe:

When asking a sender where it will complete, *tell it where it will start*.

That is done by passing the receiver's environment when asking the sender for its completion domain. Instead of `get_domain(get_env(sndr))`, the query would be `get_domain(get_env(sndr), get_env(rcvr))` (but with a query other than `get_domain`, read on).

That change has some ripple effects, the biggest of which is that the receiver is not known during early customization. Therefore, early customization is irreparably broken and must be removed.

> There are no algorithms in `std::execution` that are affected by the removal of early customization since they all do their work lazily. Should a future algorithm be added that eagerly connects a sender, that algorithm should accept

> an optional execution environment by which users can provide the starting domain. That is not onerous.

There are other ripples from the proposed change. They are described in full detail in section Fixing algorithm customization.

There are risks with trying to fix the problem now. It is a design change happening uncomfortably close to the release of C++26. One mitigating factor is that the major Standard Library vendors seem to be in no rush to implement `std::execution`. If there are lingering problems, they could be fixed with the usual DR process.

This fix has been implemented in NVIDIA's CCCL library since mid-September 2025 (see NVIDIA/cccl#5793) and in stdexec since late November (see NVIDIA/stdexec#1683). At the time of writing (early January, 2026), the changes have not resulted in any bug reports.

§ 5 Fixing algorithm customization

Selecting the right implementation of an algorithm requires requires two things:

1. Identifying the starting and completing domain of the algorithm's async operation, and
2. Using that information to select the preferred implementation for the algorithm that operation represents.

Let's take these two separately.

§ 5.1 Determining the starting and completing domains

As described in Fix algorithm customization now, so-called "early" customization, which determines the return type of `then(sndr, fn)` for example, is irreparably broken. It needs the sender to know where it will complete, which it can't in general.

So the first step is to remove early customization. There is no plan to add it back later.

That leaves "late" customization, which is performed by the `connect` customization point. The receiver, which is an extension of caller, knows where the operation will start. If the sender is given this information – that is, if the sender is told where it will start – it can accurately report where it will complete. This is the key insight.

When `connect` queries a sender's attributes for its completion domain, it should pass the receiver's environment. That way a sender has all available information when computing its completion domain.

§ 5.1.1 `get_completion_domain`

It is sometimes the case that a sender's value and error completions can happen on different domains. For example, imagine trying to schedule work on a GPU. If it succeeds, you are in the GPU domain, Bob's your uncle. If scheduling fails, however, the error cannot be reported on the GPU because we failed to make it there!

So asking a sender for a singular completion domain is not flexible enough.

When asking for a completion *scheduler*, we have three queries, one for each completion disposition: `get_completion_scheduler<set_[value|error|stopped]_t>`. Similarly, we should have three separate queries for a sender's completion domain: `get_completion_domain<set_[value|error|stopped]_t>`.

> **ASIDE:**
> If we have the `get_completion_scheduler` queries, why do we need `get_completion_domain`? We can ask the completion scheduler for its domain, right? The answer is that there are times when a sender's completion domain is knowable but the completion scheduler is not. E.g., `when_all(s1, s2)` completes on the completion scheduler of either `s1` or `s2`, so its completion scheduler is indeterminate. But if `s1` and `s2` have the same completion *domain*, then we know that `when_all` will complete in that domain.

The addition of the completion domain queries creates a nice symmetry as shown in the table below (with additions in green):

	Receiver	**Sender**

Query for scheduler	`get_scheduler`	`get_completion_scheduler<set_value_t>`
		`get_completion_scheduler<set_error_t>`
		`get_completion_scheduler<set_stopped_t>`

Query for domain	`get_domain`	`get_completion_domain<set_value_t>`
		`get_completion_domain<set_error_t>`
		`get_completion_domain<set_stopped_t>`

For a sender `sndr` and an environment `env`, we can get the sender's `set_value` completion domain as follows:

```
auto completion_domain = get_completion_domain<set_value_t>
      (get_env(sndr), env);
```

A sender like `just()` would implement this query as follows:

```
struct just_attrs
{
  auto query(get_completion_domain_t<set_value_t>, const auto&
       env) const noexcept
  {
    // an inline sender completes where it starts. the domain of
         the environment is where
    // the sender will start, so return that.
    return get_domain(env);
```

```
  }
  //...
};

template<class... Values>
struct just_sender
{
  //...
  auto get_env() const noexcept
  {
    return just_attrs{};
  }
  //...
};
```

> **Note:**
> A query that accepts an additional argument is novel in `std::execution`, but the query system was designed to support this usage. See 33.2.2 [exec.queryable.concept].

Most algorithms will want to dispatched based on where the operation will complete *successfully* and would thus use the `get_completion_domain<set_value_t>` query. To accommodate those algorithms that might want to dispatch differently, this paper proposes to add a fourth form for the `get_completion_domain` query: `get_completion_domain<>` (without a completion tag parameter). See Addressing feedback from LEWG design review for details about this query.

Just as the `get_completion_domain` queries accept an optional `env` argument, so too should the `get_completion_scheduler` queries.

§ 5.2 Algorithm dispatching in `connect`

With the addition of the `get_completion_domain<*>` queries that can accept the receiver's environment, `connect` can now know the starting and completing domains of the async operation it is constructing. When passed arguments `sndr` and `rcvr`, the starting domain is:

```
// Get the operation's starting domain:
auto starting_domain = get_domain(get_env(rcvr));
```

To get the completion domain:

```
// Get the operation's completion domain:
auto completion_domain = get_completion_domain<>(get_env(sndr),
        get_env(rcvr));
```

Now `connect` has all the information it needs to select the correct algorithm implementation. Great!

But this presents the `connect` function with a dilemma: how does it use *two* domains to pick *one* algorithm implementation?

Consider that the starting domain might want a say in how `start` works, and the completing domain might want a say in how `set_value` works. So should we let the starting domain customize `start` and the completing domain customize `set_value`?

No. `start` and `set_value` are bookends around an async operation; they must match. Often `set_value` needs state that is set up in `start`. Customizing the two independently is madness.

§ 5.2.1 Solving the double-dispatch problem

The solution is to use sender transforms. Each domain can apply its transform in turn. I do not have a reason to believe the order matters, but it is important that when asked to transform a sender, a domain knows whether it is the "starting" domain or the "completing" domain.

Here is how a domain might customize `bulk` when it is the completing domain:

```
struct thread_pool_domain
{
  template<sender-for<bulk_t> Sndr, class Env>
   auto transform_sender(set_value_t, Sndr&& sndr, const Env&
        env) const
  {
    //...
  }
};
```

Since it has `set_value_t` as its first argument, this transform is only applied when `thread_pool_domain` is an operation's completion domain. Had the first argument been `start_t`, the transform would only be used when `thread_pool_domain` is a starting domain.

§ 5.2.2 `transform_sender`

In this proposed design, the `connect` CPO does a few things:

1. Determines the starting and completing domains,
2. Applies the completing domain's transform (if any),
3. Applies the starting domain's transform (if any) to the resulting sender,
4. Connnects the twice-transformed sender to the receiver.

The first three steps are doing something different than connecting a sender and receiver, so it makes sense to factor them out into their own utility. As it so happens we already have such a utility: `transform_sender`.

The proposal requires some changes to how `transform_sender` operates. This new `transform_sender` still accepts a sender and an environment, but it no longer accepts a domain. It computes the two domains and applies the two transforms, recursing if a transform changes the type of the sender.

A possible implementation of `transform_sender` is listed in Appendix A: Listing for updated transform_sender.

With the definition of `transform_sender` in Appendix A, `connect(sndr, rcvr)` is equivalent to `transform_sender(sndr, get_env(rcvr)).connect(rcvr)` (except `rcvr` is evaluated only once).

§ 5.3 Revisiting the problematic example

Let's see how this new approach addresses the problems noted in the motivating example above. The troublesome code is:

```
namespace ex = std::execution;
auto sndr = ex::starts_on(gpu, ex::just()) | ex::then(fn);
std::this_thread::sync_wait(std::move(sndr));
```

An illustrative example describes how the current design and the "fixed" one proposed in [P3718R0] go off the rails while determining the domain in which the function `fn` will execute, causing it to use a CPU implementation instead of a GPU one.

In the new design, when the `then` sender is being connected to `sync_wait`'s receiver, the starting domain will still be the `default_domain`, but when asking the sender where it will complete, the answer will be different. Let's see how:

— When asked for its completion domain, the `then` sender will ask the `starts_on` sender where it will complete, as if by:

```
auto& [_, fn, child1] = *this; // *this is a then sender
                               // child1 is a starts_on
        sender
auto   dom1   =   ex::get_completion_domain<ex::set_value_t>
        (ex::get_env(child1), ex::get_env(rcvr));
```

— In turn, the `starts_on` sender asks the `just` sender where it will complete, *telling it where it will start*. (This is the new bit.) It looks like:

```
auto& [_, sch, child2] = *this;  // *this is a starts_on
        sender
                                 // child2 is a just sender
// ask for the scheduler's completion domain:
auto  sch_dom  =  ex::get_completion_domain<ex::set_value_t>
        (sch, get_env(rcvr));
// construct an env that reflects the fact that child2 will
        be started on sch:
auto env2 = ex::env{ex::prop{ex::get_scheduler, sch},
                    ex::prop{ex::get_domain, sch_dom},
                    ex::get_env(rcvr)};
```

```
// pass the new env when asking child2 for its completion
        domain:
auto   dom2   =   ex::get_completion_domain<ex::set_value_t>
        (ex::get_env(child2), env2);
```

— The `just` sender, when asked where it will complete, will respond with the domain on which it is started. In other words, `get_completion_domain<set_value_t>(get_env(just()), env2)` will return `get_domain(env2)`, which is `sch_dom`.

— Having correctly determined that the `then` sender will start on the default domain and complete on the GPU domain, `connect` can select the right implementation for the `then` algorithm. It does that as follows:

```
auto&& env = ex::get_env(rcvr);
return  ex::transform_sender(forward<Sndr>(sndr),  env).con-
        nect(forward<Rcvr>(rcvr));
```

The `transform_sender` call will execute the following (simplified):

```
ex::default_domain().transform_sender(
  ex::start,
  gpu_domain().transform_sender(ex::set_value, sndr, env),
  env)
```

where `gpu_domain` is the domain of the `gpu` scheduler. The `default_domain` does not apply any transformation to `then` senders, so this expression reduces to:

```
gpu_domain().transform_sender(ex::set_value,           sndr,
        ex::get_env(rcvr))
```

So in the new customization scheme, the GPU domain gets a crack at transforming the `then` sender before it is connected to a receiver, as it should.

§ 5.4 Customizing `continues_on` and `schedule_from`

One of the uglier parts of the current algorithm customization design is that it needs special case handling for the `continues_on` and `schedule_from` algorithms. The proposed design gives us an opportunity to clean this up significantly.

§ 5.4.1 Background

In order to transition between two execution contexts, each of which may know nothing about the other, it is necessary to do it in two steps: a transition *from* a context to the default domain, and a transition from the default domain *to* another context. A scheduler can customize either or both of these two steps by customizing the `continues_on` and `schedule_from` algorithms.

The customization of `continues_on` is found using the completion domain of `sndr`, making it the sanctioned way to transition *off of* a context. `schedule_from` finds its customization using the domain of `sch`, making it useful for transitioning *onto* a context.

When not customized, connecting the sender `continues_on(sndr, sch)` performs a switcheroo and connects `schedule_from(sch, sndr)` instead. In that way, both the source and destination contexts get a say in how the execution transfer is mediated.

What if a domain wants to customize `continues_on`? Asking `continues_on(sndr, sch)` for its completion domain will yield the domain of `sch`, but `continues_on` wants to use the completion domain of `sndr`. The usual `transform_sender` mechanism does not seem to cut it.

This is handled in the current draft wording by making `continues_on` a special case in `transform_sender`. In [P3718R0], the special casing is replaced with a `get_domain_override` attribute query, by which the `continues_on` sender can force `transform_sender` to use a different domain.

Both of these solutions are hacks.

§ 5.4.2 A simplification

A better way to solve this problem is to divide responsibilities differently between `continues_on` and `schedule_from`. Suppose that only `continues_on` transfers execution, and `schedule_from` does nothing and only exists so it can be customized. The `continues_on` customization point would look like:

```
constexpr pipeable-adaptor continues_on =
  []<class Sndr, class Sch>(this auto self, Sndr&& sndr, Sch sch)
  {
    return make-sender(self, sch, schedule_from(forward<Sndr>(sndr)));
  };
```

The `schedule_from` customization point would look like this:

```
constexpr auto schedule_from =
  []<class Sndr>(this auto self, Sndr&& sndr)
  {
    return make-sender(self, {}, forward<Sndr>(sndr));
  };
```

Semantically, `schedule_from(sndr)` is equivalent to `sndr`. Crucially, that means that `schedule_from(sndr)` has the same completion domain as `sndr`. And that makes `schedule_from` a great way to customize how to transition *off of* an execution context.

On the other hand, `continues_on(sndr, sch)` completes on the domain of `sch`, making it a great way to customize how to transition *onto* an execution context.

By splitting `continues_on` and `schedule_from` in this way, we obviate the need for any special cases or domain overrides. The usual `transform_sender` mechanism is sufficient.

In the CCCL project, I have implemented this design and ported my CUDA stream scheduler to use it. I needed to customize `schedule_from` for the CUDA stream scheduler to

mediate the execution transfer from the GPU back to the CPU. Besides the `bulk` and `let` algorithms, `schedule_from` is the only algorithm the GPU scheduler needs to customize.

> NOTE Carving the two algorithms this way flips how they are dispatched. `continues_on` now dispatches based on the domain of `sch`, and `schedule_from` on the completion domain of the predecessor sender. The original dispatching semantics were chosen arbitrarily. The author believes the new `continues_on`/`schedule_from` dispatch semantics are more sensible.

§ 5.5 `inline_scheduler` improvements

The suggestion above to extend the `get_completion_scheduler<*>` query presents an intriguing possibility for the `inline_scheduler`: the ability for it to report the scheduler on which its scheduling operations complete!

Consider the sender `schedule(inline_scheduler())`. Ask it where it completes today and it will say, "I complete on the `inline_scheduler`," which isn't terribly useful. However, if you ask it, "Where will you complete – and by the way you will be started on the `parallel_scheduler`?", now that sender can report that it will complete on the `parallel_scheduler`.

The result is that code that uses the `inline_scheduler` will no longer cause the *actual* scheduler to be hidden.

This realization is the motivation behind the change to strike the `get_completion_scheduler<set_value_t>(get_env(schedule(sch)))` requirement from the `scheduler` concept. We want that expression to be ill-formed for the `inline_scheduler`. Instead, we want the following query to be well-formed:

```
get_completion_scheduler<set_value_t>(get_env(schedule(in-
        line_scheduler())), get_env(rcvr))
```

That expression should be equivalent to `get_scheduler(get_env(rcvr))`, which says that the sender of `inline_scheduler` completes wherever it is started.

§ 5.6 Indeterminate domains

When computing completion domains, it is sometimes the case that an operation can complete on domain `A` or domain `B` for a given disposition (value, error, or stopped). Imagine such a sender with an indeterminate completion domain for `set_value`. How does algorithm customization work in that case?

First, we recognize that very few algorithms will ever be customized; a given domain may only customize a handful. Given `sndr | then(fn)`, there is no difficulty picking the implementation for `then` even if `sndr` can complete successfully on either domain `A` or `B`, *provided neither domain customizes* `then`.

That insight makes it advantageous for a sender to report *all* the domains on which it might complete for a particular completion channel. It can do that with a new domain type: `indeterminate_domain<Domains...>`, which looks like this:

```
template<class... Domains>
struct indeterminate_domain
{
  template<class Tag, class Sndr, class Env>
  static constexpr auto transform_sender(Tag, Sndr&& sndr, con-
        st Env& env)
  {
    // Mandates: for all D in Domains, the expression
     // D().transform_sender(Tag(), forward<Sndr>(sndr), env) is
        either ill-formed or else
    // has the same type as
      //  default_domain().transform_sender(Tag(),  forward<Sndr>
        (sndr), env)
         return  default_domain().transform_sender(Tag(),  for-
        ward<Sndr>(sndr), env);
  }
};
```

Given an environment `e`, a sender like `when_all(sndrs...)` would have a value completion domain of

```
COMMON-DOMAIN(COMPL-DOMAIN(set_value_t, sndrs, e)...)
```

where:

— *`COMPL-DOMAIN`*`(T, S, E)` is the type of `get_completion_domain<T>(get_env(S), E)` if that expression is well-formed, or `indeterminate_domain<>()` otherwise, and

— *`COMMON-DOMAIN`*`(Ds...)` is `common_type_t<Ds...>` if that expression is well-formed, and `indeterminate_domain<Ds...>` otherwise.

The final piece is to specialize `common_type` such that `indeterminate_domain<As...>` and `indeterminate_domain<Bs...>` have a common type of `indeterminate_domain<As..., Bs...>`, and such that `common_type_t<indeterminate_domain<As...>, D>` is `indeterminate_domain<As..., D>`.

§ 5.7 The procedure for the fix

The steps for fixing algorithm customization are detailed below.

1. Remove the uses of `transform_sender` in the sender adaptor algorithm customization points (33.9.12 [exec.adapt]). Directly return the result of calling *`make-sender`* rather than passing it to `transform_sender`.

2. Remove the exposition-only helpers:

   — *`completion-domain`* (33.9.2 [exec.snd.expos]/8-9),

— *get-domain-early* (33.9.2 [exec.snd.expos]/13), and

— *get-domain-late* (33.9.2 [exec.snd.expos]/14).

3. Add the `get_completion_domain` queries:

   — `get_completion_domain<set_value_t>`

   — `get_completion_domain<set_error_t>`

   — `get_completion_domain<set_stopped_t>`

4. Change the `get_completion_scheduler` queries to accept an optional environment argument.

5. Make the `get_domain(env)` query smarter by falling back to the current scheduler's domain if `env.query(get_domain)` is ill-formed, and falling back further to `default_domain()` if `env` does not have a current scheduler.

6. Restore the ability of `env<...>::query` to accept additional arguments.

7. Rename the current `schedule_from` algorithm to `continues_on` and change it to return *make-sender*`(continues_on, sch, schedule_from(sndr))`, where `schedule_from` is a new algorithm such that `schedule_from(sndr)` is equivalent to *make-sender*`(schedule_from, {}, sndr)`.

8. Remove the (unused) `transform_env` function and the `transform_env` members of the sender algorithm CPOs and from `default_domain`.

9. Change `transform_sender` from `transform_sender(Domain, Sndr, Env...)` to `transform_sender(Sndr, Env)`. Have it compute the sender's starting and completing domains and apply their transforms to `Sndr` as shown in Appendix A: Listing for updated transform_sender.

10. Update the usages of `transform_sender` in `connect` and `get_completion_signatures` to reflect its new signature.

11. For the `transform_sender` member functions in the sender algorithm CPOs, add `set_value_t,` in the front of their parameter list. Parameterize the `transform_sender` member in `default_domain` with a leading `Tag` parameter.

12. Add a class template `indeterminate_domain<Domains...>` as described in Indeterminate domains.

13. Update the attributes of the sender algorithms to properly report their completion schedulers and completion domains given an optional `env` argument. Also update the `inline_scheduler` and its schedule-sender to compute their completion scheduler and domain from the extra `env` argument.

14. From the `scheduler` concept, replace the required expression

```
{ auto(get_completion_scheduler<set_value_t>(get_env(sched-
        ule(std::forward<Sch>(sch))))) }
    -> same_as<remove_cvref_t<Sch>>;
```

with a semantic requirement that *if* the above expression is well-formed – which it is for the `parallel_scheduler`, the `task_scheduler`, and `run_loop`'s scheduler – then it shall compare equal to `sch`. (See inline_scheduler improvements for the motivation behind these changes.)

15. For any scheduler `sch` and completion tag `Tag`, require that the expression `get_completion_scheduler<Tag>(sch, env...)`, if it is well-formed, has the same type and value as `get_completion_scheduler<Tag>(get_env(schedule(sch)), env...)`. Do likewise for the `get_completion_domain` queries.

§ 6 Addressing feedback from LEWG design review

[P3826R2] was reviewed by LEWG at the Fall 2025 meeting in Kona, HI. Two important issues with the proposed design were raised at that time, both by Robert Leahy:

1. There is a design tension between "non-dependent" senders and algorithm customization. Can a sender be non-dependent if a customization could cause the final sender to have a different set of completion signatures?

2. The proposed algorithm dispatch mechanism hard-codes the primacy of the value channel. `transform_sender` uses a sender's `set_value` completion domain to find a customization. But some senders might want `transform_sender` to use a different domain to find a customization. How can that be expressed?

Let's take these two issues separately.

§ 6.1 Non-dependent senders vs. algorithm customization

A non-dependent sender is one whose completion signatures do not depend on the receiver's environment. Non-dependent senders were added in [P3164R4] as a way to improve the usability of `std::execution` by allowing some sender expressions to be type-checked eagerly, upon construction. `just(42)` is an example of a non-dependent sender, whereas `read_env(get_scheduler)` is dependent because the value it sends is the scheduler found in the receiver's environment.

By extension `then(just(42), fn)` is also non-dependent, whereas `then(read_env(get_scheduler), fn)` is dependent. The later expression should be well-formed unconditionally, but the former should be ill-formed if `fn` is not callable with an `int` argument, like `[]{}` for example.

But what if a customization of `then` would make `then(just(42), []{})` well-formed? Were we premature to reject the expression? Can *any* customizable algorithm ever be non-dependent?

§ 6.1.1 The fix

After lengthy discussions, Robert and I came to agree that the problem would be a non-issue if there were sufficient constraints on how a customization is allowed to change a sender's completion signatures. In particular, a customization should not change a sender's value completion signatures.

As it so happens, we are already requiring this of customizations. The following text is taken from 33.9.1 [exec.snd.general].

> 1. […] For the default implementation of the algorithm that produced `sndr`, connecting `sndr` to `rcvr` and starting the resulting operation state ([exec.async.ops]) necessarily results in the potential evaluation ([basic.def.odr]) of a set of completion operations whose first argument is a subexpression equal to `rcvr`. Let `Sigs` be a pack of completion signatures corresponding to this set of completion operations, and let `CS` be the type of the expression `get_completion_signatures<Sndr, Env>()`. Then `CS` is a specialization of the class template `completion_signatures` (33.10 [exec.cmplsig]), the set of whose template arguments is `Sigs`. […] If a user-provided implementation of the algorithm that produced `sndr` is selected instead of the default:
>
> (1.1) — Any completion signature that is in the set of types denoted by `completion_signatures_of_t<Sndr, Env>` and that is not part of `Sigs` shall correspond to error or stopped completion operations, unless otherwise specified.

Since the current wording already requires customizations to preserve the value completion signatures of the original sender, no further changes are needed.

§ 6.2 Dispatching algorithms with a domain other than the value completion domain

`transform_sender`, as proposed by this paper, would use an operation's starting and completing domain to find customizations. But *which* completing domain, the value, error, or stopped? R2 of this paper proposed to always use the `set_value` completion domain.

But consider an algorithm like `translate_error(sndr, fn)` which passes value and stopped completions through unchanged, but that transforms errors with `fn` before sending them on. This algorithm may want to dispatch based on where `fn` will be called from, which would be the error completion domain, *not* the value completion domain.

After considering the issue, I decided that I agreed with Robert that the proposed design was deficient in this respect.

§ 6.2.1 The fix

I propose that in addition to the following queries:

— `get_completion_domain<set_value_t>`

— `get_completion_domain<set_error_t>`

— `get_completion_domain<set_stopped_t>`

we also have:

— `get_completion_domain<>`.

`get_completion_domain<>(attrs, env...)` would return `attrs.query(get_completion_domain<>, env...)` if that is well-formed. Otherwise, it defaults to `get_completion_domain<set_value_t>(attrs, env...)`.

`transform_sender` would determine a sender's completion domain by querying for `get_completion_domain<>`. That way, a sender can implement that query to say, "For the purposes of algorithm dispatching, use domain `D`." And if a sender does not implement that query, the `set_value` completion domain will be used instead.

Robert Leahy agrees that this change addresses his concern. This revision adds wording to that effect.

§ 7 Proposed wording

[Editor's note: In 33.4 [execution.syn], make the following changes:]

```
… as before …

namespace std::execution {
  // [exec.queries], queries
  struct get_domain_t { unspecified };
  struct get_scheduler_t { unspecified };
  struct get_delegation_scheduler_t { unspecified };
  struct get_forward_progress_guarantee_t { unspecified };
  template<class CPO>
    struct get_completion_scheduler_t { unspecified };
  template<class CPO = void>
    struct get_completion_domain_t { unspecified };
  struct get_await_completion_adaptor_t { unspecified };

  inline constexpr get_domain_t get_domain{};
  inline constexpr get_scheduler_t get_scheduler{};
            inline      constexpr      get_delegation_scheduler_t
        get_delegation_scheduler{};
  enum class forward_progress_guarantee;
```

```
      inline    constexpr    get_forward_progress_guarantee_t
      get_forward_progress_guarantee{};
template<class CPO>
                 constexpr    get_completion_scheduler_t<CPO>
      get_completion_scheduler{};
template<class CPO = void>
                  constexpr    get_completion_domain_t<CPO>
      get_completion_domain{};
       inline    constexpr    get_await_completion_adaptor_t
      get_await_completion_adaptor{};

struct get_env_t { unspecified };
inline constexpr get_env_t get_env{};

template<class T>
  using env_of_t = decltype(get_env(declval<T>()));

// [exec.prop], class template prop
template<class QueryTag, class ValueType>
  struct prop;

// [exec.env], class template env
template<queryable... Envs>
  struct env;

// [exec.domain.indeterminate], execution domains
template<class... Domains>
  struct indeterminate_domain;

// [exec.domain.default], execution domains
struct default_domain;

… as before …

template<sender Sndr>
  using tag_of_t = see below;

// [exec.snd.transform], sender transformations
template<class Domain, sender Sndr, queryable… Env>
    requires (sizeof...(Env) <= 1)
  constexpr sender decltype(auto) transform_sender(
     Domain dom, Sndr&& sndr, const Env&... env) noexcept(see
      below);

// [exec.snd.transform.env], environment transformations
template<class Domain, sender Sndr, queryable Env>
  constexpr queryable decltype(auto) transform_env(
    Domain dom, Sndr&& sndr, Env&& env) noexcept;

// [exec.snd.apply], sender algorithm application
template<class Domain, class Tag, sender Sndr, class... Args>
  constexpr decltype(auto) apply_sender(
    Domain dom, Tag, Sndr&& sndr, Args&&... args) noexcept(see
      below);

// [exec.connect], the connect sender algorithm
```

```
  struct connect_t;
  inline constexpr connect_t connect{};

… as before …
```

[Editor's note: Before subsection 33.5.1 [exec.fwd.env], insert a new subsection with stable name [exec.queries.expos] as follows:]

§ Query utilities [exec.queries.expos]

1. [exec.queries] makes use of the following exposition-only entities.

2. For subexpressions `q` and `tag` and pack `args`, let *`TRY-QUERY`*`(q, tag, args...)` be expression-equivalent to *`AS-CONST`*`(q).query(tag, args...)` if that expression is well-formed, and *`AS-CONST`*`(q).query(tag)` otherwise.

3. For subexpressions `q` and `tag` and pack `args`, let *`HIDE-SCHED`*`(q)` be an object `o` such that `o.query(tag, args...)` is ill-formed when the decayed type of `tag` is `get_scheduler_t` or `get_domain_t`, and `o.query(tag, args...)` otherwise.

[Editor's note: Change [exec.get.domain] as follows:]

§ `execution::get_domain` [exec.get.domain]

1. `get_domain` asks a queryable object for its associated execution domain tag.

2. The name `get_domain` denotes a query object. For a subexpression `env`, `get_domain(env)` is expression-equivalent to *`MANDATE-NOTHROW`*`(D())`, where `D` is the type of the first of the following expressions that is well-formed [Editor's note: Reformatted as a list.]

(2.1) — ~~*`MANDATE-NOTHROW`*`(`~~`auto(`*`AS-CONST`*`(env).query(get_domain))`

(2.2) — `get_completion_domain<set_value_t>(get_scheduler(env),` *`HIDE-SCHED`*`(env))`

(2.3) — `default_domain()`

3. `forwarding_query(execution::get_domain)` is a core constant expression and has value `true`.

[Editor's note: Change subsection 33.5.6 [exec.get.scheduler] as follows:]

1. `get_scheduler` asks a queryable object for its associated scheduler.

2. The name `get_scheduler` denotes a query object. For a subexpression `env`, `get_scheduler(env)` is expression-equivalent to

```
get_completion_scheduler<set_value_t>(MANDATE-
        NOTHROW(AS-CONST(env).query(get_sched-
        uler)), HIDE-SCHED(env))
```

Mandates: If the expression above is well-formed, its type satisfies `scheduler`.

3. `forwarding_query(execution::get_scheduler)` is a core constant expression and has value `true`.

[Editor's note: Change subsection 33.5.9 [exec.get.compl.sched] as follows:]

§ `execution::get_completion_scheduler` [exec.get.compl.sched]

1. `get_completion_scheduler<`*completion-tag*`>` obtains the completion scheduler associated with a completion tag from a sender's attributes.

2. The name `get_completion_scheduler` denotes a query object template. For a subexpression `q` and pack `envs`, the expression `get_completion_scheduler<`*completion-tag*`>(q, envs...)` is ill-formed if *completion-tag* is not one of `set_value_t`, `set_error_t`, or `set_stopped_t`. Otherwise, `get_completion_scheduler<`*completion-tag*`>(q, envs...)` is expression-equivalent to

```
~~MANDATE-NOTHROW(AS-CONST(q).query(get_comple-~~
~~        tion_scheduler<completion-tag>))~~
```

(3.1) — *MANDATE-NOTHROW*(*RECURSE-QUERY*(*TRY-QUERY*(q, get_completion_scheduler<*completion-tag*>, envs...), envs...)) if that expression is well-formed.

(3.2) — Otherwise, `auto(q)` if the type of `q` satisfies `scheduler` and `sizeof...(envs) != 0` is `true`.

(3.3) — Otherwise, `get_completion_scheduler<`*completion-tag*`>(q, envs...)` is ill-formed.

Mandates: If ~~the expression above~~ `get_completion_scheduler<`*completion-tag*`>(q, envs...)` is well-formed, its type satisfies `scheduler`.

3. For a type `Tag`, subexpression `sndr`, and pack `env`, let `CS` be `completion_signatures_of_t<decay_t<decltype((sndr))>, decltype((env))...>`. If both `get_completion_scheduler<Tag>(get_env(sndr), env...)` and `CS` are well-formed and `CS().`*count-of*`(Tag()) == 0` is `true`, the program is ill-formed.

4. Let *completion-fn* be a completion function (33.3 [exec.async.ops]); let *completion-tag* be the associated completion tag of *completion-fn*; let `args` and `envs` be ~~a~~ packs of subexpressions; and let `sndr` be a subexpression such that `sender<decltype((sndr))>` is `true` and `get_completion_scheduler<`*completion-tag*`>(get_env(sndr), envs...)` is well-formed and denotes a scheduler `sch`. If an asynchronous operation created by connecting `sndr` with a receiver `rcvr` causes the evaluation of *completion-fn*`(rcvr, args...)`, the behavior is undefined unless the evaluation happens on an execution agent that belongs to `sch`'s associated execution resource.

5. The expression `forwarding_query(get_completion_scheduler<`*completion-tag*`>)` is a core constant expression and has value `true`.

[Editor's note: After subsection 33.5.9 [exec.get.compl.sched], add a new subsection with stable name [exec.get.compl.domain] as follows.]

§ `execution::get_completion_domain` [exec.get.compl.domain]

1. `get_completion_domain<`*completion-tag*`>` obtains the completion domain associated with a completion tag from a sender's attributes.

2. The name `get_completion_domain` denotes a query object template. For a subexpression `o` and pack `env`, the expression `get_completion_domain<`*completion-tag*`>(o, env...)` is ill-formed if *completion-tag* is not one of `void`, `set_value_t`, `set_error_t`, or `set_stopped_t`. Otherwise, `get_completion_domain<`*completion-tag*`>(o, env...)` is expression-equivalent to *MANDATE-NOTHROW*`(D())`, where `D` is:

(2.1) — The type of *TRY-QUERY*`(o, get_completion_domain<`*completion-tag*`>, env...)` if that expression is well-formed.

(2.2) — Otherwise, the type of `get_completion_domain<set_value_t>(o, env...)` if *completion-tag* is `void`.

(2.3) — Otherwise, the type of *TRY-QUERY*`(get_completion_scheduler<`*completion-tag*`>(o, env...), get_completion_domain<set_value_t>, env...)` if that expression is well-formed.

(2.4) — Otherwise, `default_domain` if `scheduler<decltype((o))> && sizeof...(env) != 0` is `true`.

(2.5) — Otherwise, `get_completion_domain<`*completion-tag*`>(o, env...)` is ill-formed.

3. For a type `Tag`, subexpression `sndr`, and pack `env`, let `CS` be `completion_signatures_of_t<decay_t<decltype((sndr))>,`

`decltype((env))...>`. If both `get_completion_domain<Tag>(get_env(sndr), env...)` and `CS` are well-formed and `CS().count-of(Tag()) == 0` is `true`, the program is ill-formed.

4. Let *`completion-fn`* be a completion function ([exec.async.ops]); let *`completion-tag`* be the associated completion tag of *`completion-fn`*; let `args` and `env` be packs of subexpressions; and let `sndr` be a subexpression such that `sender<decltype((sndr))>` is `true` and `get_completion_domain<`*`completion-tag`*`>(get_env(sndr), env...)` is well-formed and denotes a domain `D`. If an asynchronous operation created by connecting `sndr` with a receiver `rcvr` causes the evaluation of *`completion-fn`*`(rcvr, args...)`, the behavior is undefined unless the evaluation happens on an execution agent of an execution resource whose associated execution domain tag is `D`.

5. The expression `forwarding_query(get_completion_domain<`*`completion-tag`*`>)` is a core constant expression and has value `true`.

[Editor's note: In 33.6 [exec.sched], change paragraphs 1, 5, and 6 as follows:]

1. The `scheduler` concept defines the requirements of a scheduler type (33.3 [exec.async.ops]). `schedule` is a customization point object that accepts a scheduler. A valid invocation of `schedule` is a schedule-expression.

```
namespace std::execution {
  template<class Sch>
    concept scheduler =
              derived_from<typename  remove_cvre-
        f_t<Sch>::scheduler_concept,      sched-
        uler_t> &&
      queryable<Sch> &&
      requires(Sch&& sch) {
         { schedule(std::forward<Sch>(sch)) } ->
        sender;
        { auto(get_completion_scheduler<set_val-
        ue_t>(
              get_env(schedule(std::forward<Sch>
        (sch))))) }
              -> same_as<remove_cvref_t<Sch>>;
      } &&
         equality_comparable<remove_cvref_t<Sch>>
        &&
      copyable<remove_cvref_t<Sch>>;
}
```

… as before …

5. For a given scheduler expression `sch`, if the expression `get_completion_scheduler<set_value_t>(get_env(schedule(sch)))` is well-formed, it shall compare equal to `sch`.

6. For a given scheduler expression `sch` , type `T`, and pack of subexpressions `env`, ~~if the expression `get_domain(sch)` is well-formed, then the expression `get_domain(get_env(schedule(sch)))` is also well-formed and has the same type.~~ the following two expressions are either both ill-formed, or both well-formed with the same type:

(6.1) — `get_completion_domain<T>(sch, env...)`

(6.2) — `get_completion_domain<T>(get_env(schedule(sch)), env...)`

Likewise, the following two expressions are either both ill-formed, or both well-formed with the same type and value:

(6.3) — `get_completion_scheduler<T>(sch, env...)`

(6.4) — `get_completion_scheduler<T>(get_env(schedule(sch)), env...)`

7. *… as before …*

[Editor's note: Change 33.9.1 [exec.snd.general] as follows:]

1. Subclauses 33.9.11 [exec.factories] and 33.9.12 [exec.adapt] define customizable algorithms that return senders. Each algorithm has a default implementation. Let `sndr` be the result of an invocation of such an algorithm or an object equal to the result (18.2 [concepts.equality]), and let `Sndr` be `decltype((sndr))`. Let `rcvr` be a receiver of type `Rcvr` with associated environment `env` of type `Env` such that `sender_to<Sndr, Rcvr>` is `true`. For the default implementation of the algorithm that produced `sndr`, connecting `sndr` to `rcvr` and starting the resulting operation state (33.3 [exec.async.ops]) necessarily results in the potential evaluation (6.3 [basic.def.odr]) of a set of completion operations whose first argument is a subexpression equal to `rcvr`. [Editor's note: Broken into a separate paragraph:]

2. Let `Sigs` be a pack of completion signatures corresponding to this set of completion operations, and let `CS` be the type of the expression `get_completion_signatures<Sndr, Env>()`. Then `CS` is a specialization of the class template `completion_signatures` (33.10 [exec.cmplsig]), the set of whose template arguments is `Sigs`. If none of the types in `Sigs` are dependent on the type `Env`, then the expression `get_completion_signatures<Sndr>()` is well-formed and its type is `CS`.

3. Each completion operation can potentially be evaluated on one of several different execution agents as determined by the semantics of the algorithm, the environment of the receiver, and the completions of any child senders. For a completion tag `T`, let $\texttt{Cs}_T$ be the set of domain tags associated with the execution agents that could potentially evaluate any of the operation's completions with tag `T`, and let $\texttt{Ds}_T$ be a pack corresponding to $\texttt{Cs}_T$. If there are no potentially evaluated completion operations

with tag type `T`, then `get_completion_domain<T>(get_env(sndr), env)` is ill-formed; otherwise, it has type *`COMMON-DOMAIN`*`<`$\texttt{Ds}_T$`...>` (33.9.2 [exec.snd.expos]).

[*Example:* Let `S` be the sender `then(sndr, fn)`. `S` has the same `set_value` completion domain as `sndr`, but if `fn`'s evaluation is potentially throwing, `S`'s `set_error` completion domain would be the *`COMMON-DOMAIN`* of `sndr`'s value and error completion domains, in accordance with the semantics of the `then` algorithm (33.9.12.9 [exec.then]). — *end example*]

4. If `sndr` can determine that all of its completion operations with tag `T` happen on execution agents associated with a particular scheduler `S` (as determined by the semantics of the algorithm, the environment of the receiver, and the completion schedulers of any child senders), then `get_completion_scheduler<T>(get_env(sndr), env)` is well-formed and has the type and value of `S`; otherwise, it is ill-formed.

   [*Example:* Let `S` be the sender from the example above. The `set_value` completion scheduler of `S` is the `set_value` completion scheduler of `sndr`, if any. But `S` can only report a `set_error` completion scheduler when invocations of `fn` are not potentially throwing or when `sndr` has no `set_error` completions. When `fn` can throw, `S` could complete with `set_error` either by forwarding an error completion from `sndr` or by completing with the exception thrown by `fn`, which would happen on an agent associated with `sndr`'s `set_value` completion scheduler. — *end example*]

[Editor's note: Broken into a separate paragraph:]

5. If a user-provided implementation of the algorithm that produced `sndr` is selected instead of the default:

(1.1) — Any completion signature that is in the set of types denoted by `completion_signatures_of_t<Sndr, Env>` and that is not part of `Sigs` shall correspond to error or stopped completion operations, unless otherwise specified.

(1.2) — If none of the types in `Sigs` are dependent on the type `Env`, then `completion_signatures_of_t<Sndr>` and `completion_signatures_of_t<Sndr, Env>` shall denote the same type.

6. Various function templates in subclause 33.9 [exec.snd] can throw an exception of type *`unspecified-exception`*. Each such exception object is of an unspecified type such that a handler of type `exception` matches (14.4 [except.handle]) the exception object but a handler of type `dependent_sender_error` does not.

   [*Note:* There is no requirement that two such exception objects have the same type. — *end note*]

[Editor's note: Change 33.9.2 [exec.snd.expos] paragraph 3 as follows:]

3. For a query object `q` ~~and~~, a subexpression `v`, and a pack of subexpressions `as`, *`MAKE-ENV`*`(q, v)` is an expression `env` whose type satisfies *`queryable`* such that the result of `env.query(q, as...)` has a value equal to `v` (18.2 [concepts.equality]). Unless otherwise stated, the object to which `env.query(q, as...)` refers remains valid while `env` remains valid.

[Editor's note: Before 33.9.2 [exec.snd.expos] paragraph 6, add two new paragraphs as follows:]

6. For a pack of subexpressions `domains`, *`COMMON-DOMAIN`*`(domains...)` is expression-equivalent to `common_type_t<decltype(auto(domains))...>()` if that expression is well-formed, and `indeterminate_domain<Ds...>()` otherwise, where `Ds` is the pack of types consisting of `decltype(auto(domains))...` with duplicate types removed.

7. For a type `Tag`, subexpression `sndr`, and pack `env`, *`COMPL-DOMAIN`*`(Tag, sndr, env)` is expression-equivalent to `D()` where `D` is the type of `get_completion_domain<Tag>(get_env(sndr), env...)` if that expression is well-formed or if `sizeof...(env) == 0` is `true`, and `indeterminate_domain()` otherwise.

[Editor's note: Change 33.9.2 [exec.snd.expos] paragraph 6 (renumbered to 8) about *`SCHED-ATTRS`* and *`SCHED-ENV`* as follows:]

8. ~~For a scheduler `sch`, *`SCHED-ATTRS`*`(sch)` is an expression `o1` whose type satisfies *`queryable`* such that `o1.query(get_completion_scheduler<Tag>())` is an expression with the same type and value as `sch` where `Tag` is one of `set_value_t` or `set_stopped_t`, and such that `o1.query(get_domain)` is expression-equivalent to `sch.query(get_domain)`.~~ *`SCHED-ENV`*`(sch)` is an expression `o2` whose type satisfies *`queryable`* such that `o2.query(get_scheduler)` is a prvalue with the same type and value as `sch`, and such that `o2.query(get_domain)` is expression-equivalent to `sch.query(get_domain)`.

[Editor's note: Remove the prototype of the exposition-only *`completion-domain`* function just before 33.9.2 [exec.snd.expos] paragraph 8, and with it remove paragraphs 8 and 9, which specify the function's behavior.]

[Editor's note: Remove 33.9.2 [exec.snd.expos] paragraphs 13 and 14 and the prototypes for the *`get-domain-early`* and *`get-domain-late`* functions.]

[Editor's note: After 33.9.2 [exec.snd.expos] paragraph 26, change the declaration of the exposition-only class *`default-impls`* as follows:]

```
struct default-impls {                                          // exposi-
      tion only
  static constexpr auto get-attrs = see below;                  // exposi-
      tion only
  static constexpr auto get-env = see below;                    // exposi-
      tion only
  static constexpr auto get-state = see below;                  // exposi-
      tion only
  static constexpr auto start = see below;                      // exposi-
      tion only
  static constexpr auto complete = see below;                   // exposi-
      tion only

  template<class Sndr, class... Env>
    static consteval void check-types();                        // exposi-
      tion only
};
```

[Editor's note: Strike 33.9.2 [exec.snd.expos] paragraph 35 as follows:]

35. ~~The member *default-impls::get-attrs* is initialized with a callable object equivalent to the following lambda:~~

```
~~[](const auto&, const auto&... child) noexcept ->
        > decltype(auto) {
  if constexpr (sizeof...(child) == 1)
    return (FWD-ENV(get_env(child)), ...);
  else
    return env<>();
}~~
```

[Editor's note: After 33.9.2 [exec.snd.expos] paragraph 47 (*not-a-sender*), add the following new paragraph]

48.
```
struct not-a-scheduler {
  using scheduler_concept = scheduler_t;

  constexpr auto schedule() const noexcept {
    return not-a-sender();
  }
};
```

[Editor's note: Add the following new paragraphs after 33.9.2 [exec.snd.expos] paragraph 50 as follows:]

```
template<class Fn, class Default, class... Args>
   constexpr auto call-with-default(Fn&& fn, Default&& value,
        Args&&... args) noexcept(see below);
```

52. Let e be the expression `std::forward<Fn>(fn)(std::forward<Args>(args)...)` if that expression is well-formed; otherwise, it is `static_cast<Default>(std::forward<Default>(value))`.

53. *Returns*: e.

54. *Remarks*: The expression in the `noexcept` clause is `noexcept(e)`.

55.
```
template<class Tag>
  struct inline-attrs {
    see below
  };
```

56. For a subexpression `env`, `inline-attrs<Tag>{}.query(get_completion_scheduler<Tag>, env)` is expression-equivalent to `get_scheduler(env)`.

57. For a subexpression `env`, `inline-attrs<Tag>{}.query(get_completion_domain<Tag>, env)` is expression-equivalent to `get_domain(env)`.

[Editor's note: Add a new subsection before [exec.domain.default] with stable name [exec.domain.indeterminate] as follows:]

§ 33.9.? `execution::indeterminate_domain` [exec.domain.indeterminate]

1.

```
namespace std::execution {
  template<class... Domains>
    struct indeterminate_domain {
      indeterminate_domain() = default;
      constexpr indeterminate_domain(auto&&) noexcept {}

      template<class Tag, sender Sndr, queryable Env>
                    static constexpr sender decltype(auto)
        transform_sender(Tag, Sndr&& sndr, const Env& env)
          noexcept(see below);
    };
}
```

```
template<class Tag, sender Sndr, queryable Env>
   static constexpr sender decltype(auto) transform_sender(Tag,
        Sndr&& sndr, const Env& env)
    noexcept(see below);
```

2. *Mandates*: For each type `D` in `Domains...`, the expression `D().transform_sender(Tag(), std::forward<Sndr>(sndr), env)` is either ill-formed or has the same decayed type as `default_domain().transform_sender(Tag(), std::forward<Sndr>(sndr), env)`.

3. *Returns*: `default_domain().transform_sender(Tag(), std::forward<Sndr>(sndr), env)`

4. *Remarks*: For a pack of types `Ds`, `common_type_t<indeterminate_domain<Domains...>, indeterminate_domain<Ds...>>` is indetermi-

nate_domain<Us...> where Us is the pack of types in Domains..., Ds... except with duplicate types removed. For a type D that is not a specialization of indeterminate_domain, common_type_t<indeterminate_domain<Domains...>, D> is D if sizeof...(Domains) == 0 is true, and common_type_t<indeterminate_domain<Domains...>, indeterminate_domain<D>> otherwise.

[Editor's note: Change 33.9.5 [exec.domain.default] as follows:]

§ 33.9.5 `execution::default_domain` [exec.domain.default]

1.

```
namespace std::execution {
  struct default_domain {
    template<class Tag, sender Sndr, queryable... Env>
        requires (sizeof...(Env) <= 1)
          static constexpr sender decltype(auto) transfor-
        m_sender(Tag, Sndr&& sndr, const Env&... env)
        noexcept(see below);

    template<sender Sndr, queryable Env>
                  static constexpr queryable decltype(auto)
        transform_env(Sndr&& sndr, Env&& env) noexcept;

    template<class Tag, sender Sndr, class... Args>
       static constexpr decltype(auto) apply_sender(Tag, Sndr&&
        sndr, Args&&... args)
        noexcept(see below);
  };
}
```

```
template<class Tag, sender Sndr, queryable... Env>
    requires (sizeof...(Env) <= 1)
   static constexpr sender decltype(auto) transform_sender(Tag,
        Sndr&& sndr, const Env&... env)
    noexcept(see below);
```

2. Let e be the expression

```
tag_of_t<Sndr>().transfor-
        m_sender(Tag(),std::forward<Sndr>(sndr),
        env...)
```

if that expression is well-formed; otherwise, static_cast<Sndr>(std::forward<Sndr>(sndr)). [Editor's note: See https://cplusplus.github.io/LWG/issue4368 for why the static_cast is necessary.]

3. *Returns*: e.

4. *Remarks*: The exception specification is equivalent to noexcept(e).

~~template<sender Sndr, *queryable* Env>~~
~~constexpr *queryable* decltype(auto) transform_env(Sndr&& sndr, Env&& env) noexcept;~~

5. ~~Let e be the expression~~

~~tag_of_t<Sndr>~~
~~().transform_env(std::forward<Sndr>~~
~~(sndr), std::forward<Env>(env))~~

~~if that expression is well-formed; otherwise, *FWD-ENV*(std::forward<Env>(env)).~~

6. ~~*Mandates*: noexcept(e) is true.~~

7. ~~*Returns*: e.~~

```
template<class Tag, sender Sndr, class... Args>
constexpr   decltype(auto)   apply_sender(Tag,   Sndr&&   sndr,
        Args&&... args)
  noexcept(see below);
```

8. Let e be the expression

```
Tag().apply_sender(std::forward<Sndr>(sndr),
        std::forward<Args>(args)...)
```

9. *Constraints*: e is a well-formed expression.

10. *Returns*: e.

11. *Remarks*: The exception specification is equivalent to `noexcept(e)`.

[Editor's note: Change 33.9.6 [exec.snd.transform] as follows:]

§ `execution::transform_sender` [exec.snd.transform]

```
namespace std::execution {
  template<~~class Domain,~~ sender Sndr, queryable~~...~~ Env>
      ~~requires (sizeof...(Env) <= 1)~~
       constexpr  sender  decltype(auto)  transform_sender(~~Domain~~
        ~~dom,~~ Sndr&& sndr, const Env&~~...~~ env)
      noexcept(see below);
}
```

1. For a subexpression `s`, let *`domain-for`*`(start, s)` be `D()` where `D` is the decayed type of `get_domain(env)` if that expressions that is well-formed, and `default_domain` otherwise.

2. Let *`domain-for`*`(set_value, s)` be `D()` where `D` is the decayed type of `get_completion_domain<>(get_env(sndr), env)` if that is well-formed, and `default_domain` otherwise.

3. Let *transformed-sndr*(dom, tag, s) be the expression

```
~~dom.transform_sender(std::forward<Sndr>(sndr),
        env...)~~
dom.transform_sender(tag, s, env)
```

if that expression is well-formed; otherwise,

```
~~default_domain().transform_sender(std::forward<Sndr>
        (sndr), env...)~~
default_domain().transform_sender(tag, s, env)
```

Let ~~*final-sndr*~~ *transform-recurse*(dom, tag, s) be the expression *transformed-sndr*(dom, tag, s) if *transformed-sndr*(dom, tag, s) and ~~sndr~~ s have the same type ignoring cv-qualifiers; otherwise, it is the expression ~~transform_sender(dom, *transformed-sndr*, env...)~~ *transform-recurse*(dom2, tag, s2) where s2 is *transformed-sender*(dom, tag, s) and dom2 is *domain-for*(tag, s2).

Let *tmp-sndr* be the expression

```
transform-recurse(domain-for(set_value,    sndr),
        set_value, sndr)
```

and let *final-sndr* be the expression

```
transform-recurse(domain-for(start,    tmp-sndr),
        start, tmp-sndr)
```

4. *Returns*: *final-sndr*.

5. *Remarks*: The exception specification is equivalent to noexcept(*final-sndr*).

[Editor's note: Remove section 33.9.7 [exec.snd.transform.env].]

[Editor's note: Change 33.9.9 [exec.getcomplsigs] paragraphs 1 and 4 as follows:]

```
template<class Sndr, class... Env>
   consteval auto get_completion_signatures() -> valid-comple-
        tion-signatures auto;
```

1. Let *except* be an rvalue subexpression of an unspecified class type *Except* such that move_constructible<*Except*> && derived_from<*Except*, exception> is true. Let *CHECKED-COMPLSIGS*(*e*) be *e* if *e* is a core constant expression whose type satisfies *valid-completion-signatures*; otherwise, it is the following expression: (*e*, throw *except*, completion_signatures()) Let *get-complsigs*<Sndr, Env...>() be expression-equivalent to remove_reference_t<Sndr>::template get_completion_signatures<Sndr, Env...>(). Let NewSndr be Sndr if sizeof...(Env)

`== 0` is `true`; otherwise, `decltype(s)` where `s` is the following expression:

```
transform_sender(
   ~~get_domain_late(declval<Sndr>(), declval<Env>()...),~~
  declval<Sndr>(),
  declval<Env>()...)
```

2. *… as before …*

3. *… as before …*

4. Given a type `Env`, if `completion_signatures_of_t<Sndr>` and `completion_signatures_of_t<Sndr, Env>` are both well-formed, ~~they shall denote the same type~~ the latter shall be a superset of the former, with completion signatures that are in `completion_signatures_of_t<Sndr, Env>` but not `completion_signatures_of_t<Sndr>` corresponding to error or stopped completion operations.

[Editor's note: Change 33.9.10 [exec.connect] paragraph 2 as follows:]

2. The name `connect` denotes a customization point object. For subexpressions `sndr` and `rcvr`, let `Sndr` be `decltype((sndr))` and `Rcvr` be `decltype((rcvr))`, let `new_sndr` be the expression

```
transform_sender(~~decltype(get_domain_late(sndr, get_env(rcvr))){},~~ sndr, get_env(rcvr))
```

and let `DS` and `DR` be `decay_t<decltype((new_sndr))>` and `decay_t<Rcvr>`, respectively.

[Editor's note: Remove 33.9.11.1 [exec.schedule] paragraph 4 as follows:]

4. ~~If the expression~~

```
~~get_completion_scheduler<set_value_t>(get_env(sch.schedule())) == sch~~
```

~~is ill-formed or evaluates to `false`, the behavior of calling `schedule(sch)` is undefined.~~

[Editor's note: Change 33.9.12.1 [exec.adapt.general] paragraph 3.2-3 as follows:]

(3.2) — A parent sender (33.3 [exec.async.ops]) with a single child sender `sndr` has an associated attribute object equal to *`FWD-ENV`*`(get_env(sndr))` (33.5.1 [exec.fwd.env]) modulo the handling of the `get_completion_scheduler<`*`completion-tag`*`>` and `get_completion_domain<`*`completion-tag`*`>` queries as described in [exec.snd.general].

(3.3) — A parent sender with more than one child sender has an associated attributes object equal to `env<>{}` modulo the handling of the `get_completion_scheduler<completion-tag>` and `get_completion_domain<completion-tag>` queries as described in [exec.snd.general].

[Editor's note: Change 33.9.12.5 [exec.starts.on] paragraphs 3 and 4, and insert a new paragraph 5 as follows:]

3. Otherwise, the expression `starts_on(sch, sndr)` is expression-equivalent to:

```
transform_sender(
        query_with_default(get_domain, sch,
      default_domain()),
  make_sender(starts_on, sch, sndr))
```

except that `sch` is evaluated only once.

4. Let `out_sndr` and env be subexpressions such that `OutSndr` is `decltype((out_sndr))`. If *`sender-for`*`<OutSndr, starts_on_t>` is `false`, then the expressions `starts_on.transform_env(out_sndr, env)` and `starts_on.transform_sender(set_value, out_sndr, env)` are ill-formed; otherwise

(4.1) — `starts_on.transform_env(out_sndr, env)` is equivalent to:

```
auto&& [_, sch, _] = out_sndr;
return JOIN-ENV(SCHED-ENV(sch), FWD-ENV(env));
```

(4.2) — `starts_on.transform_sender(set_value, out_sndr, env)` is equivalent to:

```
auto&& [_, sch, sndr] = out_sndr;
return let_value(
  schedule(sch),
  continues_on(just(), sch),
    [sndr = std::forward_like<OutSndr>(sndr)]()
        mutable
     noexcept(is_nothrow_move_constructible_v<de-
        cay_t<OutSndr>>) {
    return std::move(sndr);
  });
```

[Editor's note: Remove subsection 33.9.12.6 [exec.continues.on]]

[Editor's note: Change stable name 33.9.12.7 [exec.schedule.from] to [exec.continues.on], and change the subsection as follows:]

33.9.12.~~7~~6 `execution::`~~`schedule_from`~~`continues_on` [exec.~~`schedule.from`~~`continues.on`]

1. ~~schedule_from~~continues_on schedules work dependent on the completion of a sender onto a scheduler's associated execution resource.

   ~~[*Note 1*: schedule_from is not meant to be used in user code; it is used in the implementation of continues_on. — end note]~~

2. The name ~~schedule_from~~continues_on denotes a customization point object. For some subexpressions `sch` and `sndr`, let `Sch` be `decltype((sch))` and `Sndr` be `decltype((sndr))`. If `Sch` does not satisfy scheduler, or `Sndr` does not satisfy `sender`, ~~schedule_from(sch, sndr)~~continues_on(sndr, sch) is ill-formed.

3. Otherwise, the expression ~~schedule_from(sch, sndr)~~continues_on(sndr, sch) is expression-equivalent to~~:~~ *make-sender*(continues_on, sch, schedule_from(sndr))

   ```
   ~~transform_sender(
           query-with-default(get_domain, sch, default_domain()),
     make-sender(schedule_from, sch, sndr))~~
   ```

   ~~except that sch is evaluated only once.~~

4. The exposition-only class template *impls-for* (33.9.1 [exec.snd.general]) is specialized for ~~schedule_from_t~~continues_on_t as follows:

   ```
   namespace std::execution {
     template<>
       struct impls-for<~~schedule_from_t~~continues_on_t> : default-impls {
         ~~static constexpr auto get-attrs = see below;~~
         static constexpr auto get-state = see below;
         static constexpr auto complete = see below;

       template<class Sndr, class... Env>
         static consteval void check-types();
     };
   }
   ```

5. ~~The member impls-for<schedule_from_t>::get-attrs is initialized with a callable object equivalent to the following lambda:~~

   ```
   ~~[](const auto& data, const auto& child) noexcept
       -> decltype(auto) {
     return JOIN-ENV(SCHED-ATTRS(data), FWD-ENV(get_env(child)));
   }~~
   ```

6. The member *impls-for*<~~schedule_from_t~~continues_on_t>::*get-state* is initialized with a callable object equivalent to the following

lambda:

…as before…

…as before…

12. The member *impls-for*<~~schedule_from_t~~continues_on_t>::*complete* is initialized with a callable object equivalent to the following lambda:

    …as before…

13. Let out_sndr be a subexpression denoting a sender returned from ~~schedule_from(sch, sndr)~~continues_on(sndr, sch) or one equal to such, and let OutSndr be the type decltype((out_sndr)). Let out_rcvr be *…as before…*

[Editor's note: After 33.9.12.6 [exec.continues.on], add a new subsection with stable name [exec.schedule.from] as follows:]

§ execution::schedule_from [exec.schedule.from]

1. schedule_from offers scheduler authors a way to customize how to transition off of their schedulers' associated execution contexts.

   [*Note:* schedule_from is not meant to be used in user code; it is used in the implementation of continues_on. — *end note*]

2. The name schedule_from denotes a customization point object. For some subexpression sndr, if decltype(sndr) does not satisfy sender, schedule_from(sndr) is ill-formed.

3. Otherwise, the expression schedule_from(sndr) is expression-equivalent to *make-sender*(schedule_from, {}, sndr).

[Editor's note: Change 33.9.12.8 [exec.on] as follows:]

§ execution::on [exec.on]

1. The on sender adaptor has two forms *…as before…*

2. The name on denotes a *…as before…*

3. Otherwise, if decltype((sndr)) satisfies sender, the expression on(sch, sndr) is expression-equivalent to:

```
~~transform_sender(~~
  ~~query-with-default(get_domain, sch, default_domain()),~~
  make-sender(on, sch, sndr)~~)~~
```

~~except that sch is evaluated only once.~~

4. For subexpressions `sndr`, `sch`, and `closure`, if

(4.1) — `decltype((sch))` does not satisfy `scheduler`, or

(4.2) — `decltype((sndr))` does not satisfy `sender`, or

(4.3) — `closure` is not a pipeable sender adaptor closure object (33.9.12.2 [exec.adapt.obj]),

the expression `on(sndr, sch, closure)` is ill-formed; otherwise, it is expression-equivalent to:

```
~~transform_sender(~~
  ~~get_domain_early(sndr),~~
  make_sender(on, product_type{sch, closure}, sndr)~~)~~
```

~~except that sndr is evaluated only once.~~

5. Let `out_sndr` and `env` be subexpressions, let `OutSndr` be `decltype((out_sndr))`, and let `Env` be `decltype((env))`. If *sender_for*<`OutSndr, on_t`> is `false`, then the expression~~s on.transform_env(out_sndr, env) and~~ `on.transform_sender(set_value,out_sndr, env)` ~~are~~is ill-formed.

6. ~~Otherwise: Let *not_a_scheduler* be an unspecified empty class type.~~

7. ~~The expression on.transform_env(out_sndr, env) has effects equivalent to:~~

```
~~auto&& [_, data, _] = out_sndr;~~
~~if constexpr (scheduler<decltype(data)>) {~~
~~  return JOIN_ENV(SCHED_ENV(std::forward_like<OutSndr>(data)),~~
~~    FWD_ENV(std::forward<Env>(env)));~~
~~} else {~~
~~  return std::forward<Env>(env);~~
~~}~~
```

8. Otherwise, ~~T~~the expression `on.transform_sender(set_value,out_sndr, env)` has effects equivalent to:

```
auto&& [_, data, child] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
  auto orig_sch =
     ~~query~~call_with_default(get_scheduler ~~, env~~,
       not_a_scheduler(), env);

  ~~if constexpr (same_as<decltype(orig_sch), not_a_scheduler>) {~~
    ~~return not_a_sender{};~~
  ~~} else {~~
    return continues_on(
```

```
            starts_on(std::forward_like<OutSndr>
        (data),       std::forward_like<OutSndr>
        (child)),
      std::move(orig_sch));
  }
} else {
  auto& [sch, closure] = data;
  auto orig_sch = query call-with-default(
      get_completion_scheduler<set_value_t>, not-
        a-scheduler(), get_env(child), env);
    get_env(child),
     query-with-default(get_scheduler, env, not-
        a-scheduler()));

  if constexpr (same_as<decltype(orig_sch), not-
        a-scheduler>) {
    return not-a-sender{};
  } else {
    return write_envcontinues_on(
      continues_onwrite_env(
        std::forward_like<OutSndr>(closure)(
          continues_on(
            write_env(std::forward_like<OutSndr>
        (child), SCHED-ENV(orig_sch)),
            sch)),
        orig_schSCHED-ENV(sch)),
      SCHED-ENV(sch)orig_sch);
  }
}
```

[Editor's note: Change 33.9.12.9 [exec.then] paragraphs 3 as follows:]

3. Otherwise, the expression *then-cpo*`(sndr, f)` is expression-equivalent to:

```
transform_sender(get-domain-early(sndr),    make-
        sender(then-cpo, f, sndr))
```

except that `sndr` is evaluated only once.

[Editor's note: Change 33.9.12.10 [exec.let] as follows:]

1. `let_value`, `let_error`, and `let_stopped` *… as before …*

2. For `let_value`, `let_error`, and `let_stopped`, let *set-cpo* be `set_value`, `set_error`, and `set_stopped`, respectively. Let the expression *let-cpo* be one of `let_value`, `let_error`, or `let_stopped`. For a subexpressions `sndr` and `env`, let *let-env*`(sndr, env)` be expression-equivalent to the first well-formed expression below:

(2.1) — *SCHED-ENV*`(get_completion_scheduler<`*decayed-typeof*`<`*set-cpo*`>>(get_env(sndr), `*FWD-ENV*`(env)))`

(2.2) — *MAKE-ENV*`(get_domain, get_domain(get_env(sndr)))`

(2.2) — *MAKE-ENV*(get_domain, get_completion_domain<*decayed-typeof*<*set-cpo*>>(get_env(sndr), *FWD-ENV*(env)))

(2.3) — (void(sndr), env<>{})

3. The names let_value, let_error, and let_stopped denote *… as before …*

4. Otherwise, the expression *let-cpo*(sndr, f) is expression-equivalent to:

~~*transform_sender*(*get-domain-early*(sndr), *make-sender*(*let-cpo*, f, sndr))~~

~~except that sndr is evaluated only once.~~

5. The exposition-only class *… as before …*

6. Let *receiver2* denote the following exposition-only class template:

```
namespace std::execution {
  template<class Rcvr, class Env>
  struct receiver2 {
    … as before …
  };
}
```

Invocation of the function *receiver2*::get_env returns an object e such that

(6.1) — decltype(e) models *queryable* and

(6.2) — given a query object q and pack of subexpressions as, the expression e.query(q, as...) is expression-equivalent to env.query(q, as...) if that expression is valid; otherwise, if the type of q satisfies *forwarding-query*, e.query(q, as...) is expression-equivalent to get_env(rcvr).query(q, as...); otherwise, e.query(q, as...) is ill-formed.

7. *Effects:* Equivalent to:

… as before …

where *env-t* is the pack ~~decltype(*let-cpo*.transform_env(declval<Sndr>(), declval<Env>()))~~ decltype(*JOIN-ENV*(*let-env*(declval<*child-type*<Sndr>>(), declval<Env>()), *FWD-ENV*(declval<Env>())))).

8. *impls-for*<*decayed-typeof*<*let-cpo*>>::*get-state* is initialized with a callable object equivalent to the following:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr&
        rcvr) requires see below {
  auto& [_, fn, child] = sndr;
  using fn_t = decay_t<decltype(fn)>;
      using   env_t   =   decltype(let-env(child,__
        get_env(rcvr)));
  using args_variant_t = see below;
  using ops2_variant_t = see below;

  struct state-type {
     fn_t fn;                          // exposition
         only
     env_t env;                        // exposition
         only
      args_variant_t args;             // exposition
         only
      ops2_variant_t ops2;             // exposition
         only
  };
        return    state-type{allocator-aware-for-
        ward(std::forward_like<Sndr>(fn), rcvr),
                                        let-env(child,__
        get_env(rcvr)), {}, {}};
}
```

[Editor's note: leave paragraphs 9-13 unchanged]

14. ~~Let `sndr` and `env` be subexpressions, and let `Sndr` be `decltype((sndr))`. If *`sender-for`*`<Sndr, decayed-typeof<let-cpo>>` is `false`, then the expression *`let-cpo`*`.transform_env(sndr, env)` is ill-formed. Otherwise, it is equal to:~~

    ```
    auto& [_, _, child] = sndr;
    return JOIN-ENV(let-env(child), FWD-ENV(env));
    ```

15. Let the subexpression `out_sndr` denote … *as before* …

[Editor's note: Change 33.9.12.11 [exec.bulk] paragraph 3 and 4 as follows:]

3. Otherwise, the expression *`bulk-algo`*`(sndr, policy, shape, f)` is expression-equivalent to:

    ```
    transform_sender(get-domain-early(sndr),    make-
        sender(
     bulk-algo, product-type<see below, Shape,
        Func>{policy, shape, f}, sndr))
    ```

    ~~except that `sndr` is evaluated only once.~~

    The first template argument of *`product-type`* is `Policy` if `Policy` models `copy_constructible`, and `const Policy&` otherwise.

4. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))`. If *`sender-for`*`<Sndr, bulk_t>` is `false`, then the ex-

pression `bulk.transform_sender(set_value, sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto [_, data, child] = sndr;
auto& [policy, shape, f] = data;
auto new_f = [func = std::move(f)](Shape begin, Shape end,
        auto&&... vs)
    noexcept(noexcept(f(begin, vs...))) {
  while (begin != end) func(begin++, vs...);
}
return   bulk_chunked(std::move(child),   policy,   shape,
        std::move(new_f));
```

[*Note:* This causes the `bulk(sndr, policy, shape, f)` sender to be expressed in terms of `bulk_chunked(sndr, policy, shape, f)` when it is connected to a receiver whose execution domain does not customize `bulk`. — *end note*]

[Editor's note: Change 33.9.12.12 [exec.when.all] paragraphs 2 and 3 as follows:]

2. The names `when_all` and `when_all_with_variant` denote customization point objects. Let `sndrs` be a pack of subexpressions~~,~~ and let `Sndrs` be a pack of the types `decltype((sndrs))...` ~~, and let CD be the type common_type_t<decltype(get-domain-early(sndrs))...>. Let CD2 be CD if CD is well-formed, and default_domain otherwise~~. The expressions `when_all(sndrs...)` and `when_all_with_variant(sndrs...)` are ill-formed if any of the following is `true`:

(2.1) — `sizeof...(sndrs)` is `0`, or

(2.2) — `(sender<Sndrs> && ...)` is `false`.

3. The expression `when_all(sndrs...)` is expression-equivalent to:

```
~~transform_sender(CD2(),~~     make-sender(when_all,
        {}, sndrs...)~~)~~
```

4. The exposition-only class template impls-for ([exec.snd.expos]) is specialized for when_all_t as follows:

```
namespace std::execution {
  template<>
  struct impls-for<when_all_t> : default-impls {
    ~~static constexpr auto get-attrs = see below;~~
    static constexpr auto get-env = see below;
    static constexpr auto get-state = see below;
    static constexpr auto start = see below;
    static constexpr auto complete = see below;

    template<class Sndr, class... Env>
      static consteval void check-types();
  };
}
```

[Editor's note: Remove 33.9.12.12 [exec.when.all] paragraphs 10-11 as follows:]

```
template<class Sndr, class... Env>
  static consteval void check-types();
```

8. Let `Is` be the pack of integral template arguments of the `integer_sequence` specialization denoted by *`indices-for`*`<Sndr>`.

9. *Effects*: Equivalent to: *… as before …*

10. ~~*Throws*: Any exception thrown as a result of evaluating the *Effects*, or an exception of an unspecified type derived from `exception` when `CD` is ill-formed.~~

11. ~~The member *`impls-for`*`<when_all_t>::get_attrs` is initialized with a callable object equivalent to the following lambda expression:~~

```
[](auto&&, auto&&... child) noexcept {
  if constexpr (same_as<CD, default_domain>) {
    return env<>()
  } else {
    return MAKE_ENV(get_domain, CD())
  }
  return see below;
}
```

[Editor's note: Change 33.9.12.12 [exec.when.all] paragraphs 20 and 21 as follows:]

20. The expression `when_all_with_variant(sndrs...)` is expression-equivalent to:

```
transform_sender(CD2(),      make-sender(when_all_with_variant, {}, sndrs...));
```

21. Given subexpressions `sndr` and `env`, if *`sender-for`*`<decltype((sndr)), when_all_with_variant_t>` is `false`, then the expression `when_all_with_variant.transform_sender(set_value, sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto&& [_, _, ...child] = sndr;
return      when_all(into_variant(std::forward_like<decltype((sndr))>(child))...);
```

[*Note:* This causes the `when_all_with_variant(sndrs...)` sender to become `when_all(into_variant(sndrs)...)` when it is connected with a receiver whose execution domain does not customize `when_all_with_variant`. — *end note*]

[Editor's note: Change 33.9.12.13 [exec.into.variant] paragraph 3 as follows:]

3. Otherwise, the expression `into_variant(sndr)` is expression-equivalent to:

```
~~transform_sender(get_domain_early(sndr),~~
                    make_sender(into_variant, {},
        sndr)~~);~~
```

[Editor's note: Change 33.9.12.14 [exec.stopped.opt] paragrpah 2-4 as follows:]

2. The name `stopped_as_optional` denotes a pipeable sender adaptor object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `stopped_as_optional(sndr)` is expression-equivalent to:

```
~~transform_sender(get_domain_early(sndr),~~
                    make_sender(stopped_as_option-
        al, {}, sndr)~~)~~
```

~~except that `sndr` is only evaluated once.~~

3. The exposition-only class template *… as before …*

4. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If *`sender_for`*`<Sndr, stopped_as_error_t>` is `false`, then the expression `stopped_as_error.transform_sender(set_value, sndr, env)` is ill-formed; otherwise, it is equivalent to: *… as before …*

[Editor's note: Change 33.9.12.15 [exec.stopped.err] paragrpah 1-3 as follows:]

1. `stopped_as_error` maps an input sender's stopped completion operation into an error completion operation as a custom error type. The result is a sender that never completes with stopped, reporting cancellation by completing with an error.

2. The name `stopped_as_error` denotes a pipeable sender adaptor object. For some subexpressions `sndr` and `err`, let `Sndr` be `decltype((sndr))` and let `Err` be `decltype((err))`. If the type `Sndr` does not satisfy sender or if the type `Err` does not satisfy *`movable_value`*, `stopped_as_error(sndr, err)` is ill-formed. Otherwise, the expression `stopped_as_error(sndr, err)` is expression-equivalent to:

```
~~transform_sender(get_domain_early(sndr),~~    make_
        sender(stopped_as_error, err, sndr)~~)~~
```

except that `sndr` is only evaluated once.

3. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If *`sender_for`*`<Sndr, stopped_as_error_t>` is `false`, then the expression `stopped_as_error.transform_sender(set_value, sndr, env)` is ill-formed; otherwise, it is equivalent to: *… as before …*

[Editor's note: Change 33.9.12.16 [exec.associate] paragraph 9 as follows:]

9. The name `associate` denotes a pipeable sender adaptor object. For subexpressions `sndr` and `token`:

(9.1) — If `decltype((sndr))` does not satisfy `sender`, or `remove_cvref_t<decltype((token))>` does not satisfy `scope_token`, then `associate(sndr, token)` is ill-formed.

(9.2) — Otherwise, the expression `associate(sndr, token)` is expression-equivalent to:

```
~~transform_sender(get-domain-early(sndr),~~
                    make-sender(associate, as-
        sociate-data(token, sndr))~~)~~
```

~~except that sndr is evaluated only once.~~

[Editor's note: Change 33.9.13.1 [exec.sync.wait] paragraphs 4 as follows:]

4. The name `this_thread::sync_wait` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `this_thread::sync_wait(sndr)` is expression-equivalent to the following, ~~except that sndr is evaluated only once~~:

```
apply_sender(~~get-domain-early(sndr)~~Domain(),
        sync_wait, sndr)
```

where `Domain` is the type of `get_completion_domain<set_value_t>(get_env(sndr), sync-wait-env{})`.

Mandates:

(4.1) — `sender_in<Sndr, sync-wait-env>` is `true`.

(4.2) — The type *`sync-wait-result-type`*`<Sndr>` is well-formed.

(4.3) — `same_as<decltype(e), sync-wait-result-type<Sndr>>` is `true`, where *`e`* is the `apply_sender` expression above.

[Editor's note: Change 33.9.13.2 [exec.sync.wait.var] paragraph 1 as follows:]

1. The name `this_thread::sync_wait_with_variant` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype(into_variant(sndr))`. The expression `this_thread::sync_wait_with_variant(sndr)` is ~~expression-~~equivalent to the following, except `sndr` is evaluated only once:

```
apply_sender(~~get-domain-early(sndr)~~Domain(),
        sync_wait_with_variant, sndr)
```

where `Domain` is the type of `get_completion_domain<set_value_t>(get_env(sndr), sync-wait-env{})`.

Mandates:

(1.1) — `sender_in<Sndr, sync-wait-env>` is `true`.

(1.2) — The type *`sync-wait-with-variant-result-type`*`<Sndr>` is well-formed.

(1.3) — `same_as<decltype(e), sync-wait-with-variant-result-type<Sndr>>` is `true`, where *e* is the `apply_sender` expression above.

[Editor's note: Change 33.11.1 [exec.prop] as follows:]

```
namespace std::execution {
  template<class QueryTag, class ValueType>
  struct prop {
    QueryTag query_;            // exposition only
    ValueType value_;           // exposition only

    constexpr const ValueType& query(QueryTag, auto&&...) const
        noexcept {
      return value_;
    }
  };

  template<class QueryTag, class ValueType>
            prop(QueryTag,   ValueType)   ->   prop<QueryTag,
        unwrap_reference_t<ValueType>>;
}
```

… as before …

[Editor's note: Change 33.11.2 [exec.env] as follows:]

```
namespace std::execution {
  template<queryable... Envs>
  struct env {
    Envs_0 envs_0;              // exposition only
    Envs_1 envs_1;              // exposition only
      ⋮
    Envs_n-1 envs_n-1;          // exposition only

    template<class QueryTag, class... Args>
          constexpr decltype(auto) query(QueryTag q,
        Args&&... args) const noexcept(see below);
  };

  template<class... Envs>
    env(Envs...) -> env<unwrap_reference_t<Envs>...>;
}
```

1. The class template `env` is used to construct a queryable object from several queryable objects. Query invocations on the resulting object are resolved by attempting to query each subobject in lexical order.

… as before …

```
template<class QueryTag, class... Args>
constexpr decltype(auto) query(QueryTag q, Args&&...
        args) const noexcept(see below);
```

5. Let *has-query* be the following exposition-only concept:

```
template<class Env, class QueryTag, class...
        Args>
  concept has-query =                    // expo-
        sition only
    requires (const Env& env, Args&&... args) {
        env.query(QueryTag(), std::forward<Args>
        (args)...);
    };
```

6. Let *fe* be the first element of $envs_0$, $envs_1$, … $envs_{n-1}$ such that the expression `fe.query(q, std::forward<Args>(args)...)` is well-formed.

7. *Constraints*: `(has-query<Envs, QueryTag, Args...> || ...)` is `true`.

8. *Effects*: Equivalent to: `return fe.query(q, std::forward<Args>(args)...);`

9. *Remarks*: The expression in the `noexcept` clause is equivalent to `noexcept(fe.query(q, std::forward<Args>(args)...))`.

[Editor's note: In 33.12.1.2 [exec.run.loop.types], add a new paragraph after paragraph 4 as follows:]

4. Let *sch* be an expression of type *run-loop-scheduler*. The expression `schedule(sch)` has type *run-loop-sender* and is not potentially-throwing if *sch* is not potentially-throwing.

5. For type *set-tag* other than `set_error_t`, the expression `get_completion_scheduler<set-tag>(get_env(schedule(sch))) == sch` evaluates to `true`.

[Editor's note: Change 33.13.3 [exec.affine.on] paragraphs 3 and 4 as follows:]

3. Otherwise, the expression `affine_on(sndr, sch)` is expression-equivalent to~~:~~ *make-sender*`(affine_on, sch, sndr)`.

```
transform_sender(get-domain-early(sndr),    make-
        sender(affine_on, sch, sndr))
```

~~except that `sndr` is evaluated only once.~~

4. ~~The exposition-only class template *impls-for* (33.9.2 [exec.snd.expos]) is specialized for `affine_on_t` as follows:~~

```
namespace std::execution {
  template<>
    struct impls-for<affine_on_t> : default-impls
          {
      static constexpr auto get-attrs =
          [](const auto& data, const auto& child)
          noexcept -> decltype(auto) {
            return JOIN-ENV(SCHED-ATTRS(data), FWD-
          ENV(get_env(child)));
        };
  };
}
```

[Editor's note: Change 33.13.4 [exec.inline.scheduler] paragraphs 1-3 as follows:]

1. `inline_scheduler` is a class that models `scheduler` (33.6 [exec.sched]). All objects of type `inline_scheduler` are equal. For a subexpression `sch` of type `inline_scheduler`, a query object `q`, and a pack of subexpressions `as`, the expression `sch.query(q, as...)` is expression-equivalent to *inline-attrs*`<set_value_t>().query(q, as...)`.

2. *inline-sender* is an exposition-only type that satisfies `sender`. The type `completion_signatures_of_t<`*inline-sender*`>` is `completion_signatures<set_value_t()>`.

3. Let `sndr` be an expression of type *inline-sender*, let `rcvr` be an expression such that `receiver_of<decltype((rcvr)), CS>` is `true` where `CS` is `completion_signatures<set_value_t()>`, then:

(3.1) — the expression `connect(sndr, rcvr)` has type *inline-state*`<remove_cvref_t<decltype((rcvr))>>` and is potentially-throwing if and only if `((void)sndr, auto(rcvr))` is potentially-throwing, and

~~(3.2)~~ — ~~the expression `get_completion_scheduler<set_value_t>(get_env(sndr))` has type `inline_scheduler` and is potentially-throwing if and only if `get_env(sndr)` is potentially-throwing.~~

[Editor's note: For reference: cplusplus/sender-receiver#349]

§ 8 Appendix A: Listing for updated `transform_sender`

The proposal requires some changes to how `transform_sender` operates. This new `transform_sender` still accepts a sender and an environment like the current one, but it no

longer accepts a domain. It computes the two domains, starting and completing, and applies the two transforms, recursing if a transform changes the type of the sender.

The implementation of `transform_sender` might look something like this:

```
template<class A, class B>
concept same-decayed = std::same_as<std::decay_t<A>, std::de-
        cay_t<B>>;

template<class Domain, class Tag>
struct transform-sender-recurse
{
  template<class Sndr, class Env>
  using result-t =
      decltype(Domain().transform_sender(Tag(), declval<Sndr>(),
        declval<const Env&>()));

  constexpr transform-sender-recurse(Domain, Tag) noexcept {}

  template<class Sndr, class Env>
   decltype(auto) operator()(this auto self, Sndr&& sndr, const
        Env& env)
  {
    if constexpr (!requires { typename result-t<Sndr, Env>; })
    {
       // Domain does not have a transform_sender for this sndr
        so use default_domain instead.
         return transform-sender-recurse<default_domain, Tag>()
        (forward<Sndr>(sndr), env);
    }
    else if constexpr (same-decayed<Sndr, result-t<Sndr, Env>>)
    {
       // Domain can transform the sender but its type does not
        change. End recursion.
      return Domain().transform_sender(Tag(), std::forward<Sndr>
        (sndr), env);
    }
    else if constexpr (same_as<Tag, start_t>)
    {
      // The starting domain cannot change, so recurse on Domain
         return self(Domain().transform_sender(start, std::for-
        ward<Sndr>(sndr), env), env);
    }
    else
    {
       // The type of sndr changed after being transformed, so
        the type of the completion
      // domain could change too. Recurse on the (possibly) new
        domain:
      using attrs_t  = env_of_t<result-t<Sndr, Env>>;
         using domain_t = decltype(get_completion_domain<>(de-
        clval<attrs_t>(), env));

      return transform-sender-recurse<domain_t, Tag>()(
         Domain().transform_sender(set_value, std::forward<Sndr>
        (sndr), env), env);
    }
```

```
  }
};

template<class Sndr, class Env>
auto transform_sender(Sndr&& sndr, const Env& env)
{
  auto starting_domain    = get_domain(env);
     auto   complete_domain          =   get_completion_domain<>
        (get_env(sndr), env);

   auto starting_transform = transform-sender-recurse(starting_-
         domain, start);
   auto complete_transform = transform-sender-recurse(complete_-
         domain, set_value);

        return    starting_transform(complete_transform(std::for-
         ward<Sndr>(sndr), env), env);
}
```

With this definition of `transform_sender`, `connect(sndr, rcvr)` is equivalent to `transform_sender(sndr, get_env(rcvr)).connect(rcvr)`, except that `rcvr` is evaluated only once.

§ 9 References

[P3164R4] Eric Niebler. 2025-04-28. Early Diagnostics for Sender Expressions.
https://wg21.link/p3164r4

[P3718R0] Eric Niebler. 2025-06-28. Fixing Lazy Sender Algorithm Customization, Again.
https://wg21.link/p3718r0

[P3826R2] Eric Niebler. 2025-11-07. Fix or Remove Sender Algorithm Customization.
https://wg21.link/p3826r2